

Table 1: Git vs GitHub (GitLab)

Feature	Git	GitHub or GitLab
• Definition	Distributed version control system (DVCS)	Cloud-based platform for hosting Git repositories
• Purpose	Tracks change in code over time	Provides a central location for code collaboration and sharing
• Functionality	Local version control, branching, merging, committing	Code hosting, issue tracking, pull requests, code review
• Interface	Command-line	Web-based interface
• Maintenance	Linux Foundation	Microsoft
• Cost	Free and open source	Free tier available; paid plans for additional features
• Collaboration	Primarily local; requires manual syncing	Built-in collaboration features like pull requests, issues
• Example	<i>git add, git commit, git push</i>	Creating repositories, managing issues, code reviews

If you start a new project called MyWebsite, you can initialise a Git repository by running *git init* in the project directory which creates a *.git* directory that tracks all your changes.

Commit: A commit records a snapshot of your project's files at a specific point in time. Each commit includes a unique identifier (a hash), an author, a timestamp, and a commit message describing the changes.

After adding a new feature, you would use *git add* to stage changes and *git commit -m "Add user authentication feature"* to create a commit. This records your changes and provides a message that helps explain what was done.

Branch: Branches allow you to work on different lines of development independently. They also help to manage multiple

features or fixes without interfering with the main codebase.

If you're working on a new feature called 'contact form', you can create a branch with *git branch contact-form* and

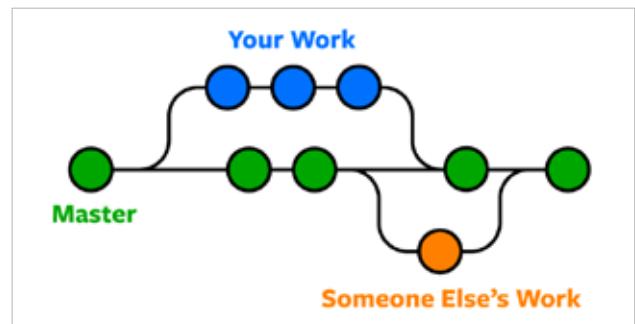


Figure 2: Branch flow (Git)

Table 2: Why Git is important

Aspect	Individual projects	Team projects
Version control	Track changes and revert versions	Track all team changes
Branching and merging	Experiment without affecting the main code	Work on features simultaneously
Collaboration	Share code and collaborate easily	Enable team collaboration
Backup and restore	Backup project to prevent data loss	Restore the project state anytime
History	Maintain change history	Log who made changes and why
Code review	Self-review code before integrating	Facilitate peer reviews
Conflict resolution	Simplify conflict resolution	Manage and resolve team conflicts
Continuous integration	Integrate with CI tools for automated testing	Support automated testing and deployment
Efficiency	Improve productivity and manage changes	Enhance team efficiency with streamlined workflows
Skill development	Encourage best practices in version control	Promote good practices and skill enhancement